



5-14-2026

Pwnagotchi: Deauthentication Attacks, WPA Handshakes, and Wireless Network Security

Emily Musgrove

Ursinus College, emmusgrove@ursinus.edu

Follow this and additional works at: https://digitalcommons.ursinus.edu/comp_hon

 Part of the [Communications Law Commons](#), [Computer Law Commons](#), and the [Computer Sciences Commons](#)

[Click here to let us know how access to this document benefits you.](#)

Recommended Citation

Musgrove, Emily, "Pwnagotchi: Deauthentication Attacks, WPA Handshakes, and Wireless Network Security" (2026). *Computer Science Honors Papers*. 3.
https://digitalcommons.ursinus.edu/comp_hon/3

This Paper is brought to you for free and open access by the Student Research at Digital Commons @ Ursinus College. It has been accepted for inclusion in Computer Science Honors Papers by an authorized administrator of Digital Commons @ Ursinus College. For more information, please contact aprock@ursinus.edu.

Pwnagotchi: Deauthentication Attacks, WPA Handshakes, and Wireless Network Security

Emily Musgrove

May 7th, 2026

Submitted to the faculty of Ursinus College in fulfillment of the requirements for Honors in
Mathematics, Computer Science, and Statistics

Abstract

Wireless networks are the foundation of modern device communication infrastructure. This project examines the security implications of automated WPA/WPA2 handshake collection using the Pwnagotchi, a portable Wi-Fi network auditing and penetration testing device. The paper provides a technical analysis of WPA and WPA2 authentication mechanisms, including the structure of the 4-way handshake, the creation of cryptographic keys, and the role of deauthentication attacks in forcing reconnections for handshake capture. Additionally, the project explores the vulnerabilities associated with using legacy wireless security protocols such as WEP and TKIP to allow for older technologies that continue to require such encryption methods. This research also investigates the machine learning architecture utilized by the Pwnagotchi, including its implementation of Long Short-Term Memory (LSTM) and an Advantage Actor-Critic (A2C) reinforcement learning model to optimize scanning and 4-way handshake capture. For this research, a functioning Pwnagotchi device was created and configured using commercially available hardware and open-source software. Beyond a technological analysis, this paper also evaluates the legal implications of widespread access to automated wireless penetration-testing devices, with a focus on their accessibility to inexperienced users and potential for misuse. Finally, this project also examines potential defensive countermeasures, including Protected Management Frames (802.11w), WPA3-SAE, network isolation, and recommended password security standards. Through this analysis, the paper argues that while devices such as the Pwnagotchi have valid educational and research purposes, their accessibility to the untrained public and ability to automate wireless network attacks raises important concerns about wireless network security and public awareness of cybersecurity.

Wireless networking technology is a vital component of modern society's communication infrastructure, which enables extensive connectivity across both personal and commercial environments. The importance of these technologies to everyday life makes the protection of wireless traffic crucial. This security is ensured by the continuously evolving Wi-Fi Protected Access (WPA) protocols. These protocols authenticate and encrypt wireless traffic, thereby certifying the confidentiality and reliability of communications between wirelessly connected devices and wireless access points. A fundamental aspect of these protocols is the 4-way handshake, a process by which both the client and the access point confirm ownership of identical key data and generate unique session keys to ensure secure, private communications.

Despite the protection provided by these protocols, they are not without their flaws. Even though WPA3 was released in 2018, WPA2 remains the most popular security protocol to date (*A beginner's guide to WPA2: Tips, use cases, & solutions*, 2025). However, despite WPA2's prevalence, the older protocol lacks defenses against many common wireless network attacks, particularly those that aim to exploit the reconnection process. One such example is deauthentication attacks, which attempt to leverage poorly authenticated management frames in order to force a client off of a network and thus retrigger the handshake process and expose sensitive key data. The public nature of 4-way handshakes, especially considering they contain inadequately encrypted data, makes them weak against potential hackers.

This project will explore the design and implementation of the Pwnagotchi, which is an automated, AI-based device created for the portable observation of wireless networks and the collection of WPA handshakes. In order to best support this goal, this project will also seek to develop a detailed technical understanding of the various mechanisms under which the Pwnagotchi operates. This explanation will focus particularly on the structure of the 4-way

handshake and the role of deauthentication attacks in forcing reconnections. Special focus will be paid on the role of these mechanisms in illegal or unethical hacker attacks, and whether widespread access to Pwnagotchi-style devices poses a potential risk to the average individual. Additionally, this project will also seek to provide a technical understanding of the AI model used by the Pwnagotchi, particularly with how its reward system is determined and its action is judged. Finally, the project will also aim to critically analyze the ethical implications and societal consequences of widespread access to understandable, automated wireless network penetration testing tools, including their potential for misuse, unintended illegal acts, and the feasibility of defensive countermeasures.

Before examining the process of automated handshake collection, it is necessary to understand the underlying security mechanism that governs the WPA and WPA2. One key process in these protocols is the 4-way handshake. The access point initiates the 4-way handshake process whenever a client associates with, or connects to, the access point (Molenaar, 2025). For the client, this process is necessary to prove that it owns a copy of the correct key material. Since the key material contains an encrypted version of the network password, this also acts to confirm that the user has provided the correct password (J., 2024). Once it is confirmed that both parties own identical copies of the correct key material, the 4-way handshake serves to establish a new session between the client and the access point as well as generate a unique session key (J., 2024).

This process of authenticating the connection and then forming the connection occurs across four messages sent between the client and the access point, thus earning the process its name. Each message involves one party sending the opposing party an Extensible Authentication Protocol over LAN (EAPOL) Key Frame, which contains a small portion of the data needed to

complete this process (Evilsocket, 2019b). In the first message, the access point sends the client its ANonce, an identifier for the access point used to confirm the uniqueness of the handshake, and sets up the precursors within the key frame for the various keys and data elements that will be transmitted in the future (J., 2024; Molenaar, 2025). Then, in the second message, the client responds to the access point by sending its own nonce, called the SNonce, and a message integrity code (MIC) (Molenaar, 2025). This code is generated using keys derived from the Pairwise Master Key (PMK), which itself contains a copy of the wireless network's password (J., 2024). By sending a copy of the MIC, the client is attempting to confirm that it has an accurate copy of the PMK. Next, in the third message, the access point sends a new message to the client, which validates the MIC sent in the previous message, derives the Pairwise Transient Key (PTK), and generates the Group Temporal Key (GTK) (Molenaar, 2025). Finally, in the fourth message, the client decrypts and installs copies of the PTK and GTK before sending a new MIC to the access point to confirm the complete installation of the keys (Molenaar, 2025). Once all four messages have been sent, the client and access point are able to securely communicate, thereby allowing the client to wirelessly broadcast data.

While the 4-way handshake does not directly transmit the wireless network password, it does expose enough of the key material to allow an attacker to verify potential password guesses offline. To understand why captured handshakes remain valuable to attackers, it is necessary to understand how WPA2 generates the PMK, PTK, and MIC for the 4-way handshake. For WPA2-Personal, the security protocol version used by most personal, home networks, a Pre-Shared Key (PSK), which is equivalent to the PMK, is used to authenticate the connection between the client and the access point (Buchanan, 2023; Kaspersky, 2025; *Network Security 2*, 2019). WPA2 calculates this key using the PBKDF2-HMAC-SHA1 function. This function operates by

combining the network's password with the "salt", a variable that is typically equivalent to the network's SSID, before hashing the value across multiple iterations using the HMAC-SHA1 hash function (Buchanan, 2023). Creating a value that is a combination of two private values, before hashing it numerous times makes it so that a hacker is not able to take the PSK and quickly reverse the process (Buchanan, 2023). The security of the key is further strengthened by the number of iterations that the function undergoes. WPA2 continues to use the standard of 4,096, which was set by Kerberos in 2005 (Buchanan, 2023; Raeburn, 2005). However, this process leaves the PSK extremely susceptible to offline dictionary and brute force attacks.

The PBKDF2-HMAC-SHA1 function is pseudorandom (Buchanan, 2023). This means that although the output of the function appears random, the function still uses a deterministic algorithm, which is entirely predictable if its seed is known (Barak, 2021). Given that WPA2 uses a combination of the network's SSID, which is publicly available information in addition to being contained in the 4-way handshake, and password to generate this seed, a hacker needs only to guess the network's password. In order to confirm if the guess is correct, the hacker would just perform a short series of steps to regenerate the MIC. Firstly, the hacker would need to regenerate the PMK using the guess, SSID, and the original values for dkLen and iterations, both of which are standardized in WPA2-Personal. Then, the resulting PMK, ANonce, SNonce, and the MAC addresses of both the access point and the client would need to be inserted into WPA2's custom Pseudo-Random Function (otherwise called PRF-512) to determine the new PTK (Praneeth, 2019). Afterwards, the hacker would take any of the EAPOL-Key Frames in the 4-way handshake that contains a copy of the original MIC and replace the MIC with an array of all zeros. In order to regenerate the MIC, the hacker would only need to run the Key Confirmation Key (KCK), which is found in the first 128 bits (or 16 bytes) of the PTK, through

the HMAC-SHA1 function with the modified EAPOL-Key Frame (Praneeth, 2019). The result of this function, truncated to the first 16 bytes, is the new value of the MIC. If this MIC is identical to the MIC received in the original 4-way handshake, the guess password is correct (Croix, 2019). As such, a hacker seeking to identify the password to the network for which it captured a 4-way handshake would only need to generate potential guesses using a brute force or dictionary attack before recalculating the MIC to determine their accuracy. While attempts to guess a network's password may typically be subject to guess limits, these attacks operate offline and as such are not punished for wrong guesses. All in all, while this method may seem tedious, it is an effective way for hackers to gain access to private networks.

While this process may seem laborious to do by hand, it can be easily automated by hackers, such that a single, weak password can be cracked in a matter of minutes. While Kerberos' recommendation to use a standard of 4,096 iterations may have been robust in 2005, technology has since rapidly advanced beyond the point at which 4,096 iterations were a robust defense against these attacks. For example, the NVIDIA GTX1080 graphics card can calculate 400,000 hashes per second (Carballal et al., 2022). Since 2023, the Open Worldwide Application Security Project has strongly recommended upgrading the standard iteration count for PBKDF2-HMAC-SHA1 from 4,096 to 1,300,000 in order to maintain security against modern technologies (*Password Storage Cheat Sheet*, 2023). This shows that current standards for wireless networks are widely ineffective for protecting against modern hackers. In fact, when researchers captured 4-way handshakes from a controlled geographic area they were able to crack 30% of the areas network passwords using only dictionary attacks on the captured 4-way handshakes (Carballal et al., 2022). The fact that researchers were able to crack these many

private network passwords shows that special concern should be given to the automated, large-scale collection of handshakes by potentially unknown individuals.

While the 4-way handshakes are insecure, some may choose to draw comfort from the fact that many private networks don't send out handshakes constantly, given that such small networks don't have users consistently disconnecting and reconnecting from the network. However, in reality, this should be of little comfort, given that a hacker can easily trigger a 4-way handshake by spoofing deauthentication frames. This behavior is referred to as a deauthentication attack and is a technique by which an attacker forces a targeted device to disconnect from a wireless network by sending forged 802.11 deauthentication frames to either the client (target) or the access point (network) (Iliadis, 2024). The attacker will compile these frames by looking for typical wireless network traffic between the client and the access point. This network traffic will contain copies of the client's and the access point's MAC addresses, which can be inserted, alongside a suitable reason code, into the deauthentication frames (*Scapy Tutorial: WiFi Security*). When these frames are sent to the device being deauthenticated, the device will quickly disconnect from the network. Attackers using deauthentication attacks typically have one of two common goals. Firstly, they may want to force the user to disconnect from a safe network in order to push them to connect to a new network, such as an Evil Twin (a fake network created by the attacker and disguised to look like the safe network), in order to collect unencrypted wireless traffic (*Evil Twin Attacks*). Secondly, they can be used to trigger the reconnection process, which includes the 4-way handshake from which they can gather potentially sensitive data (Iliadis, 2024). It is this second goal that is typically pursued by automated handshake-capture devices, such as the Pwnagotchi, since deauthentication attacks

significantly increase the trigger rate for 4-way handshakes compared to normal wireless behavior.

While most individuals are likely running WPA2 on their wireless network, it is also important to recognize the multitude of older security protocol options with their own vulnerabilities. Their recognition is important since older network security protocols are typically used as inspiration for the creation of newer security protocols, thus leading to the propagation of flaws from the older protocol in the newer protocol. One example of an older wireless security protocol that still impacts modern security vulnerabilities today is Wired Equivalent Privacy (WEP). The Wired Equivalent Privacy protocol is an obsolete security protocol first created in 1999 and introduced with the promise of a wireless network with the same security as a wired network connection (Coppock, 2025). According to Kaspersky, this protocol uses a 64- to 128-bit static key, transmitted and sent as a 10 or 26 character stored in hexadecimal, to encrypt traffic (*WEP, WPA, WPA2 and WPA3: Differences and explanation*, 2024). These keys are also shared by every device on the network, so there is no way to differentiate between users or to confirm who sends packets. In addition, WEP uses a CRC-32 checksum over plaintext before appending to it (Vyas, 2024). Finally, WEP offers two authentication methods: Open System Authentication (OSA) and Shared Key Authentication (SKA). Open System Authentication utilizes a system in which any client with the SSID can authenticate to the network, with no key needed (Vyas, 2024). Thus, there are practically no protections against hackers authenticating into the network. However, in Shared Key Authentication, the access point sends a challenge to the client, for which it is intended for them to respond with the client's WEP key (which is WEP key shared by every client on the network) (Vyas, 2024). This is remarkably less secure, because despite attempting to authenticate the client, the packets sent by the SKA method sends keystream which

reveals the vital information a bad actor would need to gain access to protected parts of the network. This security measure is incredibly insecure and hasn't been in widespread use since the early 2000s. However, WEP has remained necessary for certain devices produced in the early 2000s which remain in use today. Wireless Geographic Logging Engine (WiGLE), a site for the collection and mapping of wireless network data, showed that, as of 2026, WEP is still used by 2.54% of wireless networks studied (WiGLE). Consumers may continue to use WEP in order to allow them to connect devices like printers, IoT devices, gaming consoles, etc. which are only compatible with WEP (Vyas, 2024). A primary concern should be focused on gaming consoles only compatible with the WEP protocol. This is because, unlike legacy printers and decades-old IoT devices, which the many residential Wi-Fi users may no longer actively use and would likely willingly replace when informed of the dangers, retro gaming consoles continue to maintain a substantial presence in pop culture and many people's hearts. In fact, the global retro gaming console market was valued at 3.8 billion dollars in 2025 (Alexander, 2025). Additionally, many beloved consoles are WEP-only, including the Nintendo DS, Nintendo DS Lite, and all games built for those systems (*Compatible Wireless Modes and Wireless Security Types*). Many users may attempt to mitigate these security vulnerabilities by using WPA or WPA2-TKIP which offers to allow the user to continue to use their legacy devices on their network while promising to boost their Wi-Fi security, but this has many security concerns.

Temporal Key Integrity Protocol (TKIP) is a wireless encryption protocol designed to provide a layer of security to wireless networks while allowing for devices previously only compatible with the insecure WEP protocol. TKIP acts like a wrapper, adding vital security measures to the beginning and end of the WEP program (Awati, 2023). In doing this, TKIP adds a cryptographically generated message integrity check, an initialization vector sequencing

protocol for data transmission, per-packet key-mixing, and a re-keying tool (Awati, 2023). WPA was created by adding TKIP to WEP. While TKIP adds some security to access points, that would otherwise lack any protection, it remains a poor substitute for WPA2. The MIC used by the TKIP is only 64-bit, which is only half the size of the 128-bit MIC used by WPA2 (Awati, 2023; Kammerstetter et al., 2016). In addition, TKIP uses the same cipher that made WEP originally vulnerable (Awati, 2023). In 2010, the Wi-Fi Alliance, which manages the Wi-Fi Certified program, banned the use of TKIP- and WEP-only modes in wireless devices (Hashim, 2015). While they allowed WPA and TKIP mixed modes, they continued to highly discourage it. However, a 2013 study found that 71% of encrypted networks in the subject pool supported TKIP with 19% exclusively supporting TKIP (Vanhoef & Piessens, 2013). While this number has likely continued to drop over the years, TKIP-mixed mode remains an option even in the routers purchased for this study. While these mixed modes may allow for backward compatibility with older devices, they open a backdoor to cyber attacks by maintaining insecure encryption methods.

While the insecurity of wireless management frames does pose significant risk to the sensitive data contained therein, organizations have developed and suggested the use of specific procedures to bolster Wi-Fi security. A primary example of such a procedure are Protected Management Frames (PMF). These frames attempt to validate certain authorized management frames in order to offer a layer of protection against forged frames. The Institute of Electrical and Electronics Engineers (IEEE) implemented PMF in 2009 with the 802.11w protocol addendum, an addition to the IEEE 802 which standardized network device communications in Local Area Networks (LANs) and Metropolitan Area Networks (MLANs) (*What is IEEE 802? Networking Standards Explained (Ethernet & WIFI)*). The 802.11w protocol protects a collection of

Protected Management Frames, which offer protections against the malicious use of disassociation, deauthentication, and robust action frames for the purpose of disrupting Wi-Fi connections (Cisco, 2022; Notter, 2018). The protocol does by implementing a Integrity Group Temporal Key (IGTK), a randomly generated value provided by the access point's authentication protocol and verified by the client's authentication protocol, which acts as a signature of authenticity, thus preventing malicious actors from easy forging or spoofing these network frames with a well-known high vulnerability to misuse (Cisco, 2015). In addition to implementing the IGTK, the protocol also requires that unicast management frames must be sent via encrypted channels (Ebbecke). Without the PMF, these frames would ordinarily be sent out unprotected, which is made their forgery or spoofing significantly easier. However, given that the PMF runs on encrypted servers, the frames can only be protected after the complete initialization of the encryption process (Cisco, 2015). Thus, any frames prior to the completion of the 4-way Handshake or outside of an encrypted network, cannot be protected by the PMF. Ultimately, while PMF are imperfect, their implementation can significantly boost the security of network utilizing WPA2 or WPA3. However, despite the protections offered by implementing the 802.11w protocol, PMF has seen an underwhelming adoption. A 2021 survey of internet service providers found that only 2.22% supported Protected Management Frames (Schepers et al., 2021). This indicates that most users are likely not running PMF, despite their protections many well-known network security vulnerabilities and threats and their seventeen year endorsement by the IEEE.

Ultimately, the most important part of one's network security lies in their use of Advanced Encryption Standard, (AES). This is a security measure used with the pre-existing Pre-Shared Key and that ciphers all network traffic during transmission (*A Practical Guide to WPA2*

PSK AES Wi-Fi Security). This means even if a bad actor manages to intercept a wireless network's signal, all the packets they view will be illegible without access to the decryption cipher held by only client and access point. While AES is a security measure mandated for WPA2 and WPA3, such as how TKIP was an additional security measure added on WEP to create WPA, it does not attempt to work within the restrictions of or fight against the pre-existing flaws of the WPA2. While TKIP uses the same weak cipher as WEP, AES uses Rijndael, a symmetric block cipher, which offers a new, advanced mathematical security measure (*A Practical Guide to WPA2 PSK AES Wi-Fi Security*). AES has been the standard encryption method for the federal government for over two decades and, currently, it has no known algorithmic vulnerabilities.

The Pwnagotchi uses a Long Short-Term Memory (LSTM) with a Multilayer Perceptron (MLP) feature extractor on an Advantage Actor-Critic (A2C) agent system that runs on Bettercap, a network monitoring and penetration testing framework (Evilsocket, 2019b). While many may already have heard of a multilayer perceptron, the Pwnagotchi's use of an LSTM and an A2C agent system is somewhat advanced and not frequently seen in most AI systems. As such, this project will take the time to explain how these AI features function in order to provide more context for the functioning of the Pwnagotchi's specific AI system.

The Actor-Critic agent system is a model for reducing variance in reinforcement learning models. In such models, the "Actor" performs the actions, while the "Critic" judges the action on its progress towards the end goal. This is different from a typical reinforcement learning model in that the "Critic" judges the "Actor's" action with a specific numeric value that correlates to how good the action was (with good correlating to how well it achieved the desired goal) (*Advantage Actor-Critic (A2C)*). The Advantage Actor-Critic model takes this a step further by

using an Advantage function rather than the Action value function used by the Actor-Critic model. The Advantage function judges the value of the action not just on its own value but also how it compares to the average action value for actions at that state (*Advantage Actor-Critic (A2C)*). This incentivizes the model to choose not just a good action but rather the best action it can for that state, since the model's reward is dependent on its action value being greater than the average.

A Long Short-Term Memory is a modified version of a Recurrent Neural Network (RNN), which opts to remove the hidden state of the RNN in favor of a memory cell. While RNNs may do well at processing sequential data, they tend to overfocus on the most current data and lose focus on the past (GeeksforGeeks, 2026). This makes such models unideal for detecting long-term patterns in data. The memory cell of the LSTM operates with three gates rather than gradients. The input gate determines what data is important enough to send to the memory cell, while the forget gate determines what information is unimportant and removes it from the process (GeeksforGeeks, 2026). Once the data that made it through the input gate is processed by the memory cell, it is sent out of the output gate (GeeksforGeeks, 2026). This allows the model to include only vital information, rather than overwhelming the model with excessive, unrelated data, as RNNs are prone to do.

Ultimately, these resources offer significant benefits to the Pwnagotchi's AI model. This device operates by using Bettercap to observe Wi-Fi clients and access points, while collecting WPA key material using association and deauthentication frames (Evilsocket, 2019c). The Pwnagotchi's AI model seeks to tune its parameters, in particular those Bettercap parameters which control channel hopping and attack timings, with the intent to maximize the amount of crackable WPA key material it collects, which acts as its reward system (Evilsocket, 2019c). This

model is improved by its use of an LSTM, given that an LSTM allows for improved long-term pattern prediction in order to best predict generalizable environmental patterns within an individual wireless network environment. In addition, the A2C agent model ensures that the model determines the best parameters for optimum handshake capturing. Given the sensitivity of these parameters, the inclusion of these advanced AI systems in the Pwnagotchi's model ensures that it is capable of collecting handshakes in any environment.

While this algorithm is provided online, it is the responsibility of the user, or a 3rd party individual, to set up the device's hardware, software, and PC connection. The model I created used the five separate hardware elements: a Raspberry Pi Zero 2 WH, a 64 GB micro-SD card, a Real Time Clock (RTC) module, a PiSugar S Portable Lithium Battery, and a Waveshare eInk 2.13" Display V4. When creating this device, I selected and assembled the aforementioned materials in order to create a device that is portable, low-power, and self-contained. Therefore, these materials needed to be chosen specifically to allow the Pwnagotchi to automatically scan wireless networks for available handshakes, without requiring a continuous power source and constant physical connection to a computer-based monitoring system. The Raspberry Pi Zero 2 Wireless Header provides only enough processing power to run the Pwnagotchi image while remaining otherwise small and energy efficient. While the fact that this Pi runs as a headless setup does make it more difficult to monitor and control, particularly for those inexperienced with SSHing into such models, this setup allows for the device to be significantly more compact than a headed model by eliminating the need for a bulky monitor and keyboard. The PiSugar S Battery acts as the model's power supply, thereby allowing the model to operate without having to be connected to a fixed power source such as a wall outlet. Additionally, the RTC module allows the device to capture accurate timestamp data, which will be important for future analysis

of the data collected. Finally, the Waveshare eInk display was chosen because it is the officially supported screen of the Pwnagotchi and allows for partial screen refreshes and optimal rendering compared to other screen options (Evilsocket, 2019c). Using these supplies, the hardware setup was rather simple as the Waveshare eInk display and PiSugar Lithium Battery are able to sit on the header pins to connect to the Pi. After the materials are assembled, the software needs to be installed.

While the algorithm was premade, the process of installing it was quite difficult. Once the model was assembled, I flashed the Pwnagotchi image onto the micro-SD card using the Raspberry Pi Imager program. This image was obtained from Jayofelony's GitHub, which offers a fork of the original image (Jayofelony). The original image, created by GitHub user Evilsocket, is currently deprecated and is unusable since the necessary materials are no longer produced, and currently available materials are not supported (Evilsocket). Upon reloading the micro-SD card after its completion, I added the config.toml file, in which I was able to specify how the program is supposed to run (in regard to parameter values and acceptable behaviors), as well as a file entitled "ssh", which allowed me to SSH into the device. This process was extremely time-consuming due to the fact that the Raspberry Pi Imager failed to properly enable SSHing, and the available Pwnagotchi resources had limited information on how to solve such problems. In addition to enduring a variety of problems during the process of software installation, I also struggled considerably with the process of connecting the device to my PC.

After properly installing the software, I attempted to connect my device to my PC. While originally, I was able to physically connect the Pwnagotchi to my computer to SSH, during the research period this process became no longer possible which required me to SSH wirelessly through a router. First, I will explain the method for SSHing with a wired connection. To do this,

I, first, needed to ensure that the device maintained a consistent address, in particular a consistent Internet Protocol (IP) address. After the micro-SD card was inserted into the device, and the device was given appropriate time to boot and generate its necessary cryptographic keys, I had to manually alter its static IP address in the PC's settings while the Pi was physically connected, using a micro-USB cable, and registered in the system as a USB Ethernet/RNIS Gadget, which on my PC required the installation of the RNIS driver. For my Pwnagotchi, the IP address had to be statically set to 10.0.0.1, the subnet mask set to 255.255.255.0, the default gateway set to 10.0.0.1, and the preferred DNS server set to 8.8.8.8 (cyberspacemanmike, 2023). Then, using the PuTTY program, I was able to design the necessary SSH key, by going into the PuTTY configuration and setting at Connection>Auth>Credentials>Private key file for authentication, and SSH into the Pwnagotchi using [pi@10.0.0.2](#) (cyberspacemanmike, 2023). This allowed me to access the device, internally adjust its settings, and more broadly monitor its progress as the device searched for available 4-way handshakes.

Next, I will explain how I managed to SSH into my Pwnagotchi using a wireless connection. In this instance, wireless refers to the how the IP communications with the computer, rather than the absolute absence of cables. In fact, the first step is to connect the Pwnagotchi to the router using a micro-USB to Ethernet adapter and an Ethernet cable. Similar to with a wired connection, the next step is to ensure that the device maintains a consistent IP address. This can be done within the router through Advanced>Network>DHCP Server by enabling a DHCP Server to limit the pool of potential IP addresses to a concise list. The IP address of the device can found in the router's Connected Clients menu, in which I found the IP address of 192.168.0.209 for my Pwnagotchi. Then using the PuTTY program, I was able to use the previously generated SSH key and SSH into the Pwnagotchi using [pwnagotchi@192.168.0.209](#).

Thus, just as with the wired connection, I was able to access the internal software of the device and adjust it as needed.

Ultimately, the larger purpose of this project is to analyze the dangers of widespread access to Pwnagotchi-style devices. These devices give unaware novices and potentially dangerous hackers equal access to Wi-Fi network monitoring and penetration testing software. The Pwnagotchi is automatically set to allow for deauthentication attacks, which are illegal to perform on devices you do not own. According to 47 U.S. Code § 333, “No person shall willfully or maliciously interfere with or cause interference to any radio communication or any station licensed or authorized by or under this chapter or operated by the United States Government” (47 U.S. Code § 333 - *Willful or malicious interference*). Without modifying the device’s settings, which is not clearly explained on the device’s introductory website, the average user may be unintentionally committing a crime in their use of a small game-style device. As for those who are using the device with the intention to hack into Wi-Fi networks or otherwise create chaos, these devices and their associated resources provide excellent tongue-in-cheek explanations of how to do so (Evilsocket, 2019b). This means that innocent users would have a very hard time turning off the illegal elements, while would-be hackers are given an easy-to-understand explanation of how to do so. Thus, creating a legal lose-lose scenario for society as a whole.

In spite of the illegality concerns of Pwnagotchi and Wi-Fi penetration-testing devices, these devices have widespread availability online. In fact, a pre-built Pwnagotchi with accessories can be purchased online for as little as approximately \$110 (*Pwnagotchi - Hacker Tool* 2026). While the materials used are slightly different, this is less than the costs associated with making the Pwnagotchi used in this project. These sites are easily accessible, frequently appearing on the first page of Google search results rather than being hidden on unindexed sites.

However, given that these devices are technically legal when used properly, their accessibility is not of utmost concern but rather the lack of focus these sites place on accurately informing their consumers. While specific site I will focus on, pwnagotchi.com, does inform the consumer that these devices may run “authentication and association attacks”, they do not inform the user of the legal concern regarding those attacks (*Pwnagotchi - Hacker Tool* 2026). Additionally, they also attempt to convince the user that these devices are not as potentially dangerous as they are in reality by using words such as “cute”, “delightful”, and “friend” (*Pwnagotchi - Hacker Tool* 2026). Ultimately, while they occasionally reference the abilities of the Pwnagotchi, this site places much more focus on the fun nature of the Pwnagotchi. This, ultimately, undermines the real risks and damage that can come from using these devices without an understanding of what they do.

Given the security concerns and wide accessibility of the Pwnagotchi, this project will also take some time to focus on defensive and offensive methods that individuals and companies can take to protect networks against these devices. The best way for an average user to defend their password is to ensure that they are using a strong, secure password. While most services now require passwords to be a minimum of 8 characters and contain an uppercase letter and a symbol, studies show that many users treat this as the maximum password complexity rather than the minimum. In fact, in a collection of 239 4-way handshakes collected from 333 Wi-Fi audits, 46% of the decoded passwords were only 8 characters, and approximately 58% were solely numerical (Carballal et al., 2022). In fact, alphanumeric passwords with a combination of upper and lowercase letters accounted for only 2% of decoded passwords (Carballal et al., 2022). Additionally, once a user deems a password to be secure, they are highly likely to reuse that same password over several accounts. A 2019 security study conducted by Google showed that

52% of those surveyed used the same password across multiple accounts (Google & Harris Poll, 2019). This means that a single data leak can compromise multiple accounts per password. This data all supports the fact that the average user's password is considerably weak and vulnerable to dictionary attacks. Instead, the Cybersecurity & Infrastructure Security Agency recommends that a strong password be at least 16 characters long, with a random assortment of upper and lowercase letters, numbers, symbols, and unrelated words, with each password being unique to each account (*Require Strong Passwords*). Following these rules, along with utilizing two-factor authentication (when available) and regular password changes, will ensure that one's password is safe if the associated wireless network is attacked.

However, while ensuring you use a safe and secure password is vital, the best way for the average individual to protect bad actors from successfully attacking one's wireless network is to transition away from older encryption methods. Firstly, if a Wi-Fi router still allows for WEP encryption, it is vital that it is not used to encrypt one's wireless network considering the extreme insecurity of the encryption protocol. Similarly, while many individuals' using older technology may use TKIP or WPA, believing that it will ensure security while maintaining backward compatibility, it actually has many of the same vulnerabilities as WEP. Therefore, it is important to not use TKIP or WPA encryption for networks which you wish to ensure the security of. When willing and able to, consumers should be encouraged to replace devices that require WEP in favor of newer devices with stronger security protocols. However, when this is not possible, it is important to isolate WEP-only devices onto an exclusive and separate wireless network (*TKIP: Understanding the temporal key integrity protocol*). This maintains the security of the main network, should the less secure network be attacked. Secondly, when compatible, the most currently secure encryption method for residential Wi-Fi networks is WPA3 utilizing

Simultaneous Authentication of Equals (SAE). However, while this method is considered the most secure, it is not currently universally supported on all routers. In the event that one's router does not support WPA3, the second-best option is WPA2-PSK. While this does have the aforementioned security vulnerabilities, it is significantly more secure than WEP, TKIP, or WPA. Additionally, since AES is mandatory in WPA2 and WPA3, using these encryption methods will also provide a router with automatic access to an advanced encryption method compared to older encryption protocols (*A Practical Guide to WPA2 PSK AES Wi-Fi Security*).

In addition to private individuals altering their behavior to secure their wireless network accounts against deauthentication attacks, companies producing these wireless networks can take much greater steps to do so on behalf of them and their clients. The main method a corporation can take to protect its and its client's Wi-Fi networks is the adoption of the updated WPA3 security standard. WPA3, released in 2018, introduced SAE, which uses a unique session key for connection, rather than generating a key from a shared password as is done in WPA2 (*Are you using WPA3?*, 2025). Since this method never sends the network's password, even encrypted, the attacker is forced to actively interact with the network for each password guess (*Are you using WPA3?*, 2025). Thus, this security protocol eliminates offline brute force and dictionary attacks and makes even the online versions of such attacks wildly ineffective by forcing them to adhere to password attempt limits. Additionally, corporations that still use key hashing security protocols, such as WPA and WPA2, should adjust their standards so that these keys undergo the recommended 1,300,000 iterations, in compliance with modern recommendations. This change would prevent hackers from making as many password attempts, as each password guess would take considerably more time. Ultimately, the greatest measures for protecting against hackers

cracking a network's password come down to adhering to best practices for both password and encryption complexity.

In conclusion, now that a working model of the Pwnagotchi has been created and research has been conducted on the handshake data gathered, future research will focus on identifying the best methods for defending against Pwnagotchis. These devices have a relatively low cost of entry and pre-built models are easily accessible. As such, it is important to continue to research what information these devices will collect and how these devices and their data could, potentially, be harmful, whether or intentional or unintentionally so. Little research has been conducted on these game-ified hacking devices or their effectiveness in collecting potentially sensitive data. As such, future research will continue to focus on quantitatively evaluating the success rate of Pwnagotchi's in capturing valid WPA/WPA2 handshakes under controlled, legal conditions, as well as analyzing the types of data collected during its operation. This research will also seek to examine the effectiveness of potential defensive countermeasures in limiting automated handshake-harvesting.

References

- Advantage Actor-Critic (A2C)*. Advantage Actor-Critic (A2C) · Hugging Face. (n.d.).
<https://huggingface.co/learn/deep-rl-course/en/unit6/advantage-actor-critic>
- Alexander, R. (2025, September 17). *Retro Gaming Console Sale Statistics (2025)*. Co-op Board Games. <https://coopboardgames.com/statistics/retro-gaming-console-sales-resurgence/>
- Awati, R. (2023, April 3). *Temporal Key Integrity Protocol (TKIP)*. TechTarget.
<https://www.techtarget.com/searchmobilecomputing/definition/TKIP>
- Barak, B. (2021, November 17). *Pseudorandom functions*. An intensive introduction to cryptography. https://intensecrypto.org/public/lec_04_pseudorandom-functions.html
- A beginner's guide to WPA2: Tips, use cases, & solutions*. Meter. (2025, February 24).
<https://www.meter.com/resources/wpa2>
- Buchanan, B. (2023, October 30). *The Great WiFi and Password Protector: PBKDF2*. Medium.
<https://medium.com/asecuritysite-when-bob-met-alice/the-great-wifi-and-password-protector-pbkdf2-92aa5802f2c4>
- Carballal, A., Galego-Carro, J. P., Rodriguez-Fernandez, N., & Fernandez-Lozano, C. (2022, December 16). *Wi-Fi Handshake: Analysis of password patterns in Wi-Fi Networks*. PeerJ. Computer science. <https://pmc.ncbi.nlm.nih.gov/articles/PMC10280184/#table-2>

CISA. (n.d.). *Require Strong Passwords*. Cybersecurity & Infrastructure Security Agency.

<https://www.cisa.gov/audiences/small-and-medium-businesses/secure-your-business/require-strong-passwords>

Cisco. (2015, July 15). *802.11r, 802.11k, and 802.11w Deployment Guide, Cisco IOS-XE Release 3.3*.

https://www.cisco.com/c/en/us/td/docs/wireless/controller/technotes/5700/software/release/ios_xe_33/11rkw_DeploymentGuide/b_802point11rkw_deployment_guide_cisco_ios_xe_release33/b_802point11rkw_deployment_guide_cisco_ios_xe_release33_chapter_0100.html

Cisco. (2022, April 12). *Cisco Catalyst 9800 Series Wireless Controller Software Configuration Guide, Cisco IOS XE 17.6.x*.

https://www.cisco.com/c/en/us/td/docs/wireless/controller/9800/17-6/configuration/b_wl_17_6_cg/m_vewlc_802_11w.html

Coppock, J. (2025, September 1). *Wi-Fi Security Comparisons: What Are WEP, WPA, WPA2, and WPA3?*. Avast Academy. <https://www.avast.com/c-wep-vs-wpa-or-wpa2#wep>

Cornell. (n.d.). *47 U.S. Code § 333 - willful or malicious interference*. Legal Information Institute. <https://www.law.cornell.edu/uscode/text/47/333>

Croix, A. (2019, July 30). *How does WPA/WPA2 WIFI security work, and how to crack it?*.

[cylab.be. https://cylab.be/blog/32/how-does-wpawpa2-wifi-security-work-and-how-to-crack-it#:~:text=the%20SSID%20change,-,4%2Dway%20handshake,packets%20transmitted%20by%20the%20client](https://cylab.be/blog/32/how-does-wpawpa2-wifi-security-work-and-how-to-crack-it#:~:text=the%20SSID%20change,-,4%2Dway%20handshake,packets%20transmitted%20by%20the%20client)

cyberspacemanmike. (2023, June 24). *Pwnagotchi Tutorial, Pt. 2: Software (using Waveshare V3 and where to find RDNIS drivers)*. YouTube.

https://www.youtube.com/watch?v=7nj5Euo5Bng&list=PL2xB7IbHP_82xC931zH5iHIjLc50Q5-87&index=2

Ebbecke, P. (n.d.). *Protected Management Frames enhance Wi-Fi® network security*. Wi-Fi Alliance. <https://www.wi-fi.org/beacon/philipp-ebbecke/protected-management-frames-enhance-wi-fi-network-security>

Evil Twin Attacks. Bitdefender. (n.d.). <https://www.bitdefender.com/en-us/business/infozone/what-is-evil-twin-attack>

Evilsocket. (2019a, October 9). *Installation*. Pwnagotchi. <https://pwnagotchi.ai/installation>

Evilsocket. (2019b, October 9). *Introduction*. Pwnagotchi. <https://pwnagotchi.ai/intro/#wifi-handshakes-101>

Evilsocket. (2019c, October 9). *Usage*. Pwnagotchi. <https://pwnagotchi.ai/usage/>

Evilsocket. (n.d.). *Releases · evilsocket/pwnagotchi*. GitHub. <https://github.com/evilsocket/pwnagotchi/releases>

GeeksforGeeks. (2026, April 14). *What is LSTM - long short term memory?*

<https://www.geeksforgeeks.org/deep-learning/deep-learning-introduction-to-long-short-term-memory/>

Google, & Harris Poll. (2019, February). *Online Security Survey*. Google.

https://services.google.com/fh/files/blogs/google_security_infographic.pdf

Hashim, B. (2015, April 23). *Removal of TKIP from Wi-Fi® devices*. Silex Technology.

<https://www.silextechnology.com/unwired/removal-of-tkip-from-wi-fi-devices>

Iliadis, F. (2024, March 18). *Deauthentication Attacks*. Baeldung.

<https://www.baeldung.com/cs/deauthentication-attacks>

Internet Archive Wayback Machine. (2026, May 3). *Pwnagotchi - Hacker Tool*. pwnagotchi.

<https://web.archive.org/web/20260503013031/https://www.pwnagotchi.com/products/pwnagotchi>

J., S. C. (2024, October 14). *Understanding wi-fi security protocols: WEP, WPA, WPA2, and the Four way wpa handshake*. Understanding Wi-Fi Security Protocols: WEP, WPA, WPA2, and the Four Way Handshake.

https://codespindle.com/Ethical/WEP_WPA_WPA2_WPAHandshake.html

Jayofelony. (n.d.). *Jayofelony/Pwnagotchi: (🐧_🐧) - raspberry pi instrumenting bettercap for wi-fi pwning*. GitHub. <https://github.com/jayofelony/pwnagotchi>

Kammerstetter, M., Muellner, M., Burian, D., Kudera, C., & Kastner, W. (2016, May 25).

Efficient High-Speed WPA2 Brute Force Attacks using Scalable Low-Cost FPGA Clustering. Ar5iv. <https://ar5iv.labs.arxiv.org/html/1605.07819>

Kaspersky. (2024, December 26). *WEP, WPA, WPA2 and WPA3: Differences and explanation*.

Kaspersky. <https://usa.kaspersky.com/resource-center/definitions/wep-vs-wpa>

Kaspersky. (2025, November 27). *What is WPA2 (wireless protected access 2)?*

<https://www.kaspersky.com/resource-center/definitions/wireless-protected-access-2>

Lenovo. (n.d.-a). *TKIP: Understanding the temporal key integrity protocol*. Lenovo.

<https://www.lenovo.com/us/en/glossary/tkip/?orgRef=https%253A%252F%252Fwww.google.com%252F>

Molenaar, R. (2025, October 9). *WPA and WPA2 4-Way Handshake*. NetworkLessons.com.

<https://networklessons.com/wireless/wpa-and-wpa2-4-way-handshake>

Network Security 2. Berkley. (2019, July 18).

https://inst.eecs.berkeley.edu/~cs161/su19/lectures/lec13_net_2.pdf

Nintendo. (n.d.). *Compatible Wireless Modes and Wireless Security Types*. Nintendo Support.

https://en-americas-support.nintendo.com/app/answers/detail/a_id/498/~/compatible-wireless-modes-and-wireless-security-types

Notter, R. (2018, October 13). *802.11w-2009 – Protected Management Frames (PMF): Overview And Lab Tests*. dot11 exposed. <https://dot11.exposed/2018/10/13/802-11w-2009-protected-management-frames-pmf-overview-and-lab-tests/>

Open Worldwide Application Security Project. (2023). *Password Storage Cheat Sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html#pbkdf

Packetlabs. (2025, October 3). *Are you using WPA3?*. Packetlabs.

<https://www.packetlabs.net/posts/are-you-using-wpa3/>

A Practical Guide to WPA2 PSK AES Wi-Fi Security. Splash Access. (n.d.).

<https://www.splashaccess.com/wpa-2-psk-aes/>

Praneeth, K. (2019, November 9). *4-Way Hand shake , Keys generation and MIC Verification-*

WPA2. Praneeth's Blog. <https://praneethwifi.in/2019/11/09/4-way-hand-shake-keys-generation-and-mic-verification/>

Raeburn, K. (2005, February). *RFC 3962: Advanced encryption standard (AES) encryption for*

Kerberos 5. IETF Datatracker. <https://datatracker.ietf.org/doc/html/rfc3962>

Schepers, D., Ranganathan, A., & Vanhoef, M. (2021). Let Numbers Tell the Tale: Measuring

Security Trends in Wi-Fi Networks and Best Practices. *Proceedings of the 14th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, 100–105.

<https://doi.org/10.1145/3448300.3468286>

University of Toronto. (n.d.). Scapy Tutorial: WiFi Security.

https://www.cs.toronto.edu/~arnold/427/18s/427_18S/indepth/scapy_wifi/scapy_tut.html

Vanhoef, M., & Piessens, F. (2013). Practical verification of WPA-TKIP vulnerabilities.

Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security, 427–436. <https://doi.org/10.1145/2484313.2484368>

Vyas, R. (2024, October 27). *What Is Wired Equivalent Privacy (WEP)? | Wi-Fi Security Guide*.

SecureW2. <https://securew2.com/blog/what-is-wep-security>

What is IEEE 802? Networking Standards Explained (Ethernet & WIFI). Lenovo. (n.d.-b).

<https://www.lenovo.com/us/en/glossary/ieee802/?orgRef=https%253A%252F%252Fwww.google.com%252F>

WiGLE. (n.d.). Statistics. <https://wingle.net/stats>